ONE_GMS VERCEL DEPLOYMENT GUIDE

This guide summarizes the Vercel deployment approach that worked for the ONE_GMS frontend and backend.

Overview

- Deploy the backend and frontend as two separate Vercel projects.
- Backend project name: ‘one-gms-api‘
- Frontend project name: ‘one-gms‘
- Backend production API base URL: ‘https://one-gms-api.vercel.app‘
- Frontend production app URL: ‘https://one-gms.vercel.app‘

Final Architecture

- Backend lives in the ‘backend‘ folder and is deployed as a FastAPI project.
- Frontend lives in the ‘frontend‘ folder and is deployed as a Vite project.
- Frontend calls the backend through ‘VITE_API_BASE_URL‘.
- Frontend should use the backend root URL only, not Swagger docs.

Correct frontend production value:

```
VITE_API_BASE_URL=https://one-gms-api.vercel.app
```

Incorrect values:

```
VITE_API_BASE_URL=http://localhost:8000
VITE_API_BASE_URL=https://one-gms-api.vercel.app/docs
VITE_API_BASE_URL=VITE_API_BASE_URL=https://one-gms-api.vercel.app
```

Backend Deployment

1. Create the Backend Vercel Project

- Create a new Vercel project for the ‘backend‘ folder.
- Set the project root directory to ‘backend‘.
- Use the ‘FastAPI‘ framework preset if Vercel detects it.

2. Backend Build Settings

Use these settings:

- Framework Preset: ‘FastAPI‘
- Install Command: ‘pip install -r requirements.vercel.txt‘
- Build Command: ‘alembic upgrade head‘

The file ‘requirements.vercel.txt‘ is used to keep the Vercel Python runtime dependencies smaller and more deployment-friendly.

3. Backend Runtime Entry Point

The backend uses ‘backend/app.py‘ as the Vercel entry point.

Important detail:

- Vercel imports ‘app.py‘ directly by file path.
- Because of that, ‘backend/app.py‘ adds the ‘backend‘ directory to ‘sys.path‘ before importing ‘src.main‘.
- This avoids ‘ModuleNotFoundError: No module named ’src’‘.

4. Database Migrations on Vercel

The working approach was:

- run ‘alembic upgrade head‘ during the build
- let the app start after migrations finish successfully

This ensures the production database schema is up to date for the deployed backend version.

5. Mail Template Directory Fix

Vercel deploys the app under a read-only filesystem location.

That caused this runtime problem before:

- ‘OSError: [Errno 30] Read-only file system: ’/var/task/src/templates’‘

The fix that worked:

- keep ‘backend/src/templates/.gitkeep‘ so the templates folder is bundled
- update ‘backend/src/mail.py‘ so it uses the bundled templates folder if it exists
- otherwise fall back to a writable temp directory instead of trying to create folders in the deployed source tree

6. Trusted Host Fix

The backend also needed host normalization for Vercel domains.

The working fix in ‘backend/src/middleware.py‘:

- allow ‘localhost‘
- allow ‘127.0.0.1‘
- allow ‘*.vercel.app‘
- allow the hosts extracted from configured frontend and backend URLs
- allow the host from ‘VERCEL_URL‘

This prevents ‘400 Invalid host header‘ on production Vercel requests.

7. Backend Verification

These are normal checks:

- ‘https://one-gms-api.vercel.app/docs‘ should load Swagger UI
- opening ‘https://one-gms-api.vercel.app/api/v1/auth/login‘ directly in a browser returns ‘405 Method Not Allowed‘

That ‘405‘ is expected because the login endpoint is a ‘POST‘ endpoint, not a browser ‘GET‘ endpoint.

Frontend Deployment

1. Create the Frontend Vercel Project

- Create a separate Vercel project for the ‘frontend‘ folder.
- Set the project root directory to ‘frontend‘.
- Use the ‘Vite‘ framework preset.

2. Frontend Environment Variables

Use these values:

For local development in ‘frontend/.env‘:

```
VITE_API_BASE_URL=http://localhost:8000
VITE_API_VERSION=v1
VITE_APP_DEBUGGING=true
```

For the Vercel frontend production environment:

- Name: ‘VITE_API_BASE_URL‘
- Value: ‘https://one-gms-api.vercel.app‘

Important:

- do not include ‘/docs‘
- do not include the variable name inside the value field
- the Vercel value field must contain only the URL

3. Why ‘localhost‘ Worked Before

‘http://localhost:8000‘ works only when:

- the frontend is running locally on your machine

- the backend is also running locally on your machine

It does not work from a deployed Vercel site because ‘localhost‘ would then mean the visitor’s browser machine, not the deployed backend.

4. Frontend Runtime API Fix

The frontend now normalizes ‘VITE_API_BASE_URL‘ before using it.

This protects against three cases:

- local development value like ‘http://localhost:8000‘
- production value like ‘https://one-gms-api.vercel.app‘
- mistakenly pasted dashboard value like ‘VITE_API_BASE_URL=https://one-gms-api.vercel.app‘

This normalization is used in:

- ‘frontend/src/api/http.js‘
- ‘frontend/vite.config.js‘
- ‘frontend/apiBaseUrl.js‘

5. Redeploy Requirement

Vite environment variables are compiled into the frontend bundle at build time.

That means:

- changing a Vercel frontend environment variable does not change the currently deployed JavaScript
- the frontend must be rebuilt and redeployed after the env var changes

If the Vercel UI refuses to redeploy a canceled deployment, use a fresh commit.

Safe empty commit example:

```
git commit --allow-empty -m "Trigger frontend rebuild with updated API base URL"
git push origin main
```

6. Frontend Verification

After the correct frontend deployment goes live:

- login requests should go to ‘https://one-gms-api.vercel.app/api/v1/auth/login‘
- they should not go to ‘http://localhost:8000/api/v1/auth/login‘

If the browser console shows:

```
POST http://localhost:8000/api/v1/auth/login
```

the live frontend is still serving an older bundle.

If the browser console shows a malformed path like:

```
https://one-gms.vercel.app/VITE_API_BASE_URL=https://one-gms-api.vercel.app/api/...
```

the frontend env var value was entered incorrectly in Vercel or the frontend build is still outdated.

Final Working Checklist

Backend

- Vercel project root is ‘backend‘
- Install command is ‘pip install -r requirements.vercel.txt‘
- Build command is ‘alembic upgrade head‘
- ‘backend/app.py‘ adds the backend folder to ‘sys.path‘
- ‘backend/src/mail.py‘ avoids writing inside the read-only deployment tree
- ‘backend/src/templates/.gitkeep‘ exists
- ‘backend/src/middleware.py‘ allows Vercel hosts
- Swagger works at ‘https://one-gms-api.vercel.app/docs‘

Frontend

- Vercel project root is ‘frontend‘
- frontend production env uses ‘https://one-gms-api.vercel.app‘
- local ‘.env‘ can still use ‘http://localhost:8000‘
- frontend normalization code is present
- frontend is redeployed after env var changes
- login requests go to the backend domain, not localhost

Recommended Final Test

1. Open ‘https://one-gms-api.vercel.app/docs‘
2. Confirm the backend docs load
3. Open ‘https://one-gms.vercel.app/login‘
4. Open browser DevTools
5. Submit login
6. Confirm the request is:

```
POST https://one-gms-api.vercel.app/api/v1/auth/login
```

If that request appears, the deployment wiring is correct.